

Peer-to-Peer Distributed File Sharing System

Two synchronised trackers, clients that download from many peers at once, and a security layer on every connection.

Name	Shubham Kumar Sunny
Roll number	2025201025
Institute	International Institute of Information Technology, Hyderabad
Language	C++11, POSIX sockets and threads, OpenSSL
Report version	October 2026 (describes the code as it is now)

2

trackers that hold the same information

512 KB

piece size; every piece checked with SHA-1

4

pieces of one file in flight at the same time

8 + 8

worker threads: download pool and seeder pool

7.8 s

to download a 1 GB file (measured, section 10)

12 MB

peak client memory during that 1 GB download

0

memory leaks reported by the leak checker

≈ 2,800

lines of C++ in tracker and client

This report explains how the system is built and why. It is written in plain language: each section starts with the idea, shows a picture, and only then names the functions in the code. Sections 2–7 cover what the assignment asks for. Section 8 covers security work that goes beyond the assignment. Sections 9–11 cover the problems met on the way, the tests, and what is still missing.

1 The system in one page

2 Design overview

3 Network protocols

4 Files, pieces and integrity

5 The download engine and piece selection

6 Tracker synchronisation

7 Threads and locks

8 Security (beyond the assignment)

9 Challenges and how they were solved

10 Testing and results

11 Limitations and future work

12 References

Source layout: `tracker/` · `client/` · `README.md` (how to run and check everything, with screenshots) · `tools/` · `report/` (source of this report)

1 The system in one page

People form groups and share files inside a group. The files are not stored on a server. They stay on the computers of the people who have them.

There are two kinds of program.

- **The tracker** is a small server that remembers *who exists* and *who has what*: users, groups, the hashes of each shared file, and the address of every client that is online. It never touches file data. There are two trackers, so one can stop without stopping the system.
- **The client** is what a user runs. It is three things at once: a **shell** for typing commands, a **downloader** that fetches pieces of a file from several other clients at the same time, and a **seeder** that gives the pieces it has to others.

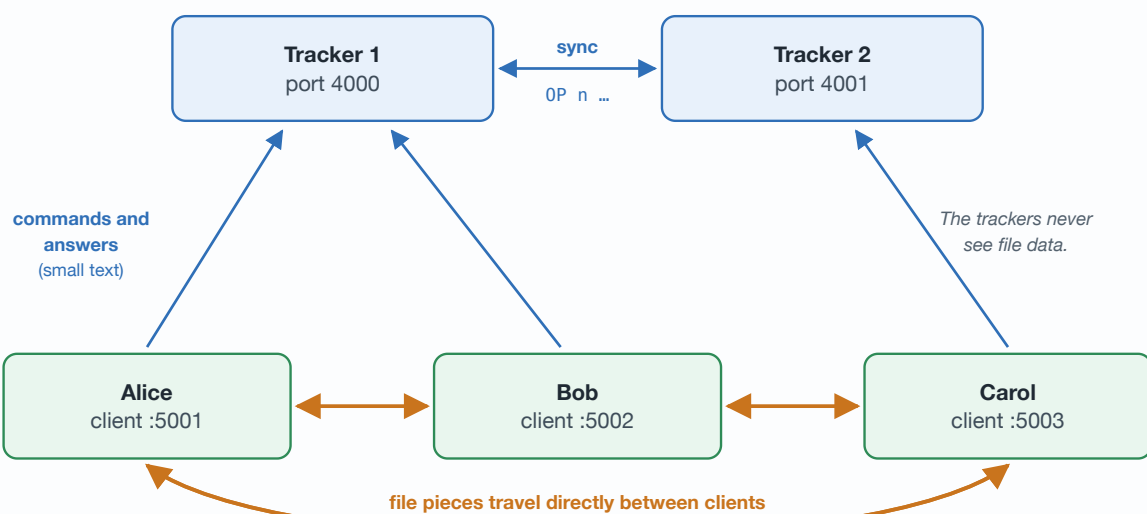


Figure 1. The whole system. Blue lines carry small text messages (accounts, groups, "who has this file?"). Orange lines carry the actual file, cut into pieces, directly from client to client.

What happens when Bob downloads a file

1. Bob types `download_file movies movie.bin .`
2. His client asks a tracker. The tracker checks that Bob is a member of the group and answers with the file size, the SHA-1 hash of the file and of every 512 KB piece, and the addresses of the clients that have it.
3. Bob's client asks each of those clients which pieces it has.
4. It fetches four pieces at a time, from different clients, the rarest pieces first. Each piece is checked against its hash as soon as it arrives. A bad piece is thrown away and fetched from someone else.
5. When all pieces are in, the whole file is hashed once more and the file appears under its final name.
6. From the first verified piece on, Bob's client is itself a seeder for that file.

How the assignment's requirements are met

The assignment asks for	How it is done	Section
Exactly two trackers with the same state	Every change is a numbered operation, written to a log and forwarded to the other tracker	6
Keeps working with one tracker; a returning tracker catches up	Clients switch trackers by themselves; missed operations are resent from the log	6
512 KB pieces; SHA-1 per piece and per file	Hashes computed at upload, kept by the tracker, checked at download	4
Files up to 1 GB without excessive memory	One 512 KB buffer at a time; 12 MB peak measured for 1 GB	4, 10
Corrupted pieces fetched again from another peer	Per-piece check, per-peer failure counters, retry queue	5
One file from several peers; several files at once	Fixed thread pool shared by all downloads, 4 pieces per file in flight	5, 7
Custom TCP protocols that survive partial sends	Three small protocols; every send and receive loops until complete	3
Thread-safe shared data; no leaks; sockets closed	Every shared structure behind a mutex; clean under ThreadSanitizer; 0 leaks measured	7, 10
The 14 commands, the launch format, <code>[C] [group_id] filename</code>	Implemented exactly as written in the assignment	2

2 Design overview

The main design decision: split "what" from "where"

Information about files (names, sizes, hashes, who has them) is tiny. The files themselves are huge. So the two are handled by different parts:

- **Small and must be consistent** → **the trackers**. Everything is kept in memory behind one lock. Simple to get right.
- **Large and must be fast** → **client to client**. The trackers are never in the path of file data, so they cannot become a bottleneck.

The modules

File	Its one job
<code>tracker/server.cpp</code>	Sockets and threads: accept connections, one thread per client, the link to the other tracker, <code>quit</code>
<code>tracker/commands.cpp</code>	The "brain": all the data, every command, the permission checks, the operation log
<code>tracker/password.cpp</code>	Turns passwords into salted, slow hashes
<code>tracker/tls.cpp</code>	TLS for the tracker's connections

File	Its one job
client/client.cpp	The shell: reads commands, talks to the tracker, switches tracker when one dies
client/peer.cpp	The seeder and the download engine (the largest and hardest file)
client/threadpool.cpp	A fixed-size pool of worker threads
client/fileutils.cpp	SHA-1 of a buffer; piece hashes and whole-file hash of a file
client/tls.cpp	TLS for the client's connections; this client's own key pair

The split inside the tracker matters: `server.cpp` knows nothing about users or groups, and `commands.cpp` knows nothing about sockets. A command arrives as a line of text and leaves as a line of text. That made it possible to replay the same commands from a log file and from the other tracker without any extra code.

The commands

All commands of the assignment are implemented with the exact names and arguments given there:

```
create_user login logout create_group join_group leave_group list_groups
list_requests accept_request upload_file list_files download_file
show_downloads stop_share

./tracker tracker_info.txt <tracker_no>          ./client <IP>:<PORT> tracker_info.txt
```

Two small additions help when demonstrating: `verbose on` prints every piece as it moves, and `help` lists the commands.

What the tracker remembers

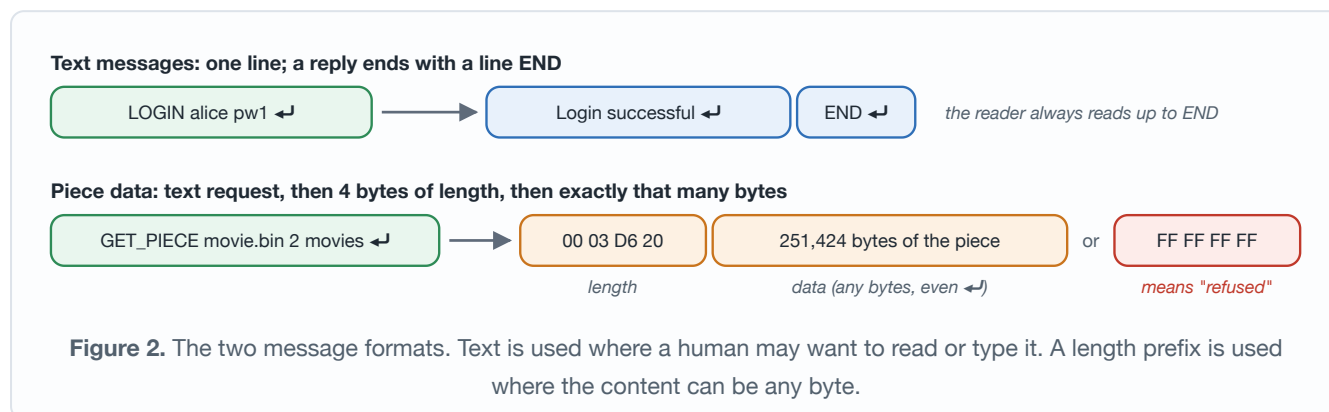
Data structure	Meaning	Why this container
<code>map<string, User> users</code>	user → password hash, logged-in flag	look-up by name
<code>map<string, string> groups</code>	group → owner	look-up by name
<code>map<string, set<string>> members</code>	group → its members	"is X a member?" is asked constantly; a set answers quickly and stays sorted
<code>map<string, vector<string>> requests</code>	group → pending join requests	kept in arrival order
<code>map<string, vector<FileMeta>> group_files</code>	group → files: size, file hash, piece hashes, set of seeders	few files per group; a list is enough
<code>map<string, string> seeder_addr</code>	user → address and key fingerprint, only while logged in	look-up by name
<code>conn_user, user_conn</code>	connection ↔ logged-in user	both directions are needed

Who you are is decided by the tracker, not by what you send. A login is tied to the connection it arrived on. Every later command on that connection is checked against that user. A client cannot act as someone else by simply writing another name into a command.

3 Network protocols

TCP delivers bytes in order, but it does not tell the receiver where one message ends and the next begins. Deciding that is called *framing*, and it is the heart of protocol design.

Two kinds of framing, each where it fits



Format	Used for	Why
A line of text; a reply ends with a line <code>END</code>	client ↔ tracker, tracker ↔ tracker, and the request line between clients	Easy to read in logs and to type by hand when debugging. The same text is written to the operation log.
4-byte length, then data	the reply that carries a piece or a bitfield	File data can contain any byte, including a newline, so "read until newline" cannot work. The receiver knows exactly how many bytes to wait for.

The three conversations

1. Client → tracker

```
client:  LOGIN alice pw1
tracker: Login successful
        END

client:  DOWNLOAD_FILE movies movie.bin
tracker: movie.bin 1300000 <file sha1> 3
        <sha1 of piece 0>
        <sha1 of piece 1>
        <sha1 of piece 2>
        127.0.0.1:5001 <alice's key fingerprint>
        END
```

The client turns what the user types into a request that names the acting user (`create_group movies` becomes `CREATE_GROUP movies alice`). The same form is used in the log and between trackers, so one parser serves all three.

2. Client → client

```
GET_BITFIELD movie.bin movies    → 00 00 00 03 '1' '1' '0'    (which pieces I have)
GET_PIECE movie.bin 2 movies      → 00 03 D6 20 <251,424 bytes>
anything not allowed              → FF FF FF FF
```

One request per connection, then close. This keeps the protocol free of state: nothing can get out of step. The price is a new connection per piece (section 11).

3. Tracker → tracker

```
TRACKER_HELLO      →   HAVE 4          ("I have your operations 1 to 4")
OP 5 CREATE_GROUP movies alice
OP 6 JOIN_GROUP movies bob              (a stream of numbered operations)
```

Status and error messages

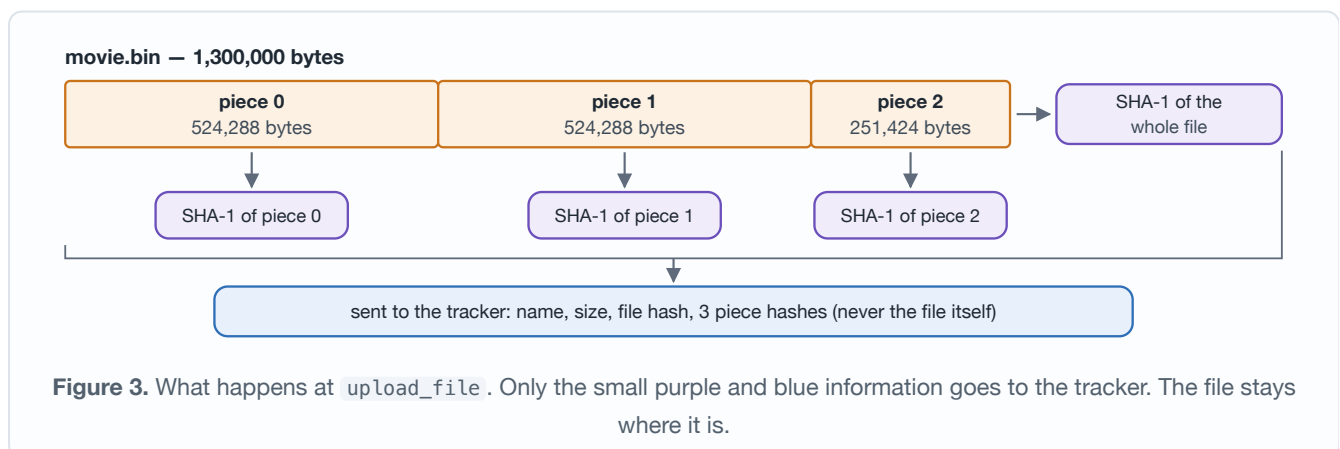
Every request gets an answer a person can read: Login failed, No group, User not in group, Only the group owner can accept requests, Permission denied: you are logged in as bob. Between clients there is exactly one error code, the length FF FF FF FF, which can never be a real piece length.

Things that go wrong on a network, and the answer to each

Problem	What the code does
A send or receive transfers only part of the data	Every send and receive is a loop that continues until the whole message is through (send_all , recv_all , read_from_socket , write_to_socket)
A peer goes silent in the middle	5-second limit on every read and write; a timeout counts as a failure of that peer
Connecting to a dead machine hangs for a minute	Non-blocking connect, wait with poll for at most 5 seconds
Writing to a closed connection kills the program (SIGPIPE)	The signal is ignored; the write returns an error that is handled
A request line that never ends	Length limits: 1 KB for a peer request, 16 MB for a tracker line
The tracker disappears	The client connects to the other one, logs in again and repeats the request

4 Files, pieces and integrity

A file is cut into pieces of 512 KB. Each piece gets its own fingerprint (a SHA-1 hash), and the whole file gets one more.



Why pieces

- **Speed:** different pieces can come from different clients at the same time.

- **Cheap repair:** if one piece arrives damaged, only that 512 KB is fetched again.
- **Sharing early:** a client that has half the file can already give that half to others.

The arithmetic

```

number of pieces = (size + 524287) / 524288      (rounded-up division)
offset of piece i = i × 524288
length of piece i = min(524288, size - i × 524288)  (only the last one can be shorter)

```

A 1 GB file has 2,048 pieces. An empty file has 0 pieces and is handled as a normal case.

Why two levels of hashing

Check	When	What it catches
SHA-1 of each piece	the moment the piece arrives, before it is written	a damaged or false piece, immediately, and tells us which peer sent it
SHA-1 of the whole file	once, after the last piece	anything the piece checks could miss, for example a piece written to the wrong place

Large files without large memory

The client never holds a file in memory. To hash a file it reads one 512 KB buffer at a time and feeds it to two hash calculations at once (the piece's and the whole file's). While downloading, each worker thread holds one piece. With 4 pieces in flight that is about 2 MB of file data, whatever the file size. The measurement in section 10 shows 12 MB in total for a 1 GB file.

Writing the downloaded file safely

Danger	What the code does
Pieces arrive in any order	The file is first created at its full size. Each piece is written at its own offset.
Several threads write the same file	<code>pwrite</code> writes at a given offset in one step. There is no shared "current position" that two threads could fight over.
A download fails half way	Pieces go into <code>name.part</code> . Only a complete, verified file is renamed to <code>name</code> . A failed download deletes its <code>.part</code> file, so a half-written file is never left behind.

5 The download engine and piece selection

This is the central part of the assignment: fetch one file from several peers at the same time, choose pieces sensibly, and keep going when peers misbehave.

Step 1: find out who has what

The tracker says which clients share the file. It does not say which *pieces* each one has, and that matters: a client that is still downloading has only some of them. So the downloader first asks every peer for its **bitfield**, one character per piece (1 = have it, 0 = do not).

Step 2: piece selection — rarest first

The pieces are sorted by how many peers have them, fewest first.

	piece 0	piece 1	piece 2	piece 3	piece 4	piece 5
Alice has	✓	✓	✓	✓	✓	✓
Bob has	✓	✓	✓	✓	–	–
Carol has	✓	✓	–	–	–	–
how many have it	3	3	2	2	1	1
download order	piece 4	piece 5	piece 3	piece 2	piece 0	piece 1

equal counts:
random order

Figure 4. Rarest-first with three peers. Pieces 4 and 5 exist on one peer only, so they are fetched first.

Why rarest first? If the only peer holding pieces 4 and 5 leaves, nobody can finish the file. Fetching those first means that very soon two clients have them. The rare pieces stop being rare. Common pieces can wait, because there will always be someone to get them from.

Why a random order among equals? If two clients start together and both see "pieces 4 and 5 are rarest", a fixed order makes both fetch piece 4 first. Then they hold the same piece and have nothing to give each other. With a random order one probably takes 4 and the other 5, and each can then serve the other.

How it is done: shuffle the list of pieces, then sort it by count with a *stable* sort. A stable sort keeps the shuffled order among pieces with equal counts. Pieces that nobody has yet go to the very end.

Step 3: fetch pieces in parallel

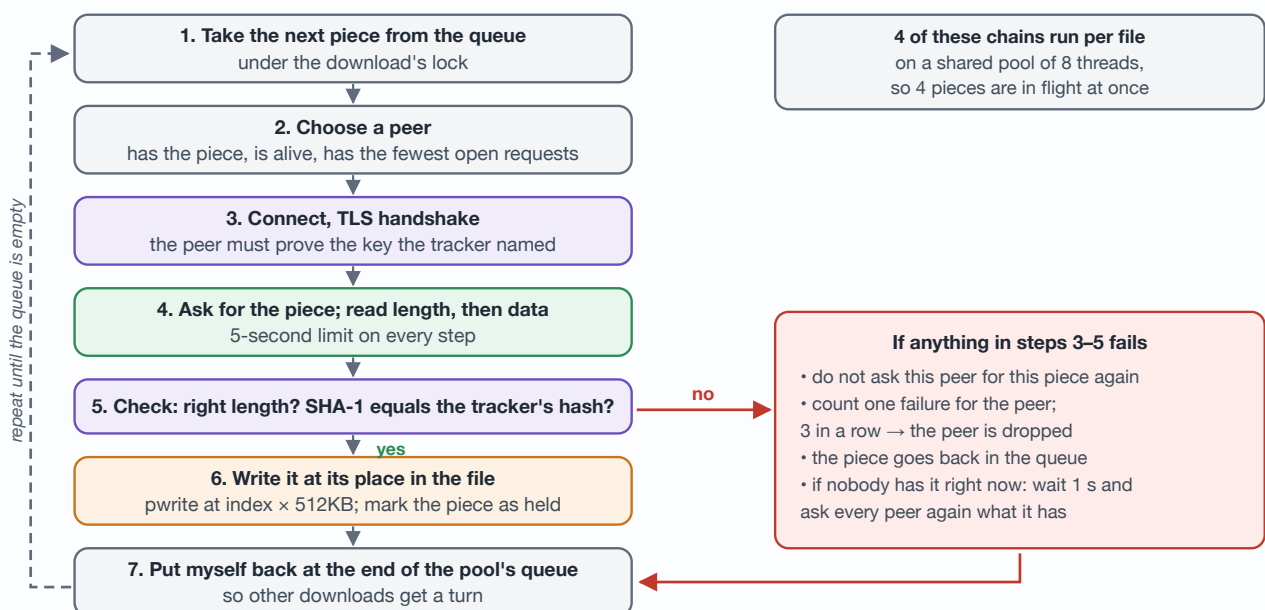


Figure 5. The life of one piece. The left column is the normal path. The red box is what happens on any failure.

Choosing the peer for a piece

Among the peers that have the piece, the one with the fewest requests currently open from this download is chosen. This spreads the work over all peers without measuring anyone's speed.

Sharing the worker threads fairly

All downloads use one pool of 8 threads. A piece task downloads *one* piece and then puts itself at the back of the pool's queue. So if three files are downloading, their tasks take turns. One huge file cannot starve a small one, and the shell stays responsive because nothing runs on its thread.

What happens when peers fail

Situation	Result
A piece arrives with a wrong hash	It is thrown away. That peer is not asked for that piece again. The piece is fetched from another peer.
A peer stops answering	Each open request fails after 5 seconds. After 3 failures in a row the peer is dropped for this download.
A peer deleted its copy	It answers "refused" to everything. It is never chosen.
Nobody has a piece right now	The task waits one second and asks all peers for their bitfields again. Peers that are still downloading gain pieces over time.
No good peer is left	The download ends as failed: [F] in <code>show_downloads</code> , the <code>.part</code> file is deleted, and the tracker is told this client no longer shares the file.

Seeding, including while still downloading

The seeder gives out only two things: files in its **share table** (files this client uploaded or finished downloading), and **verified pieces of downloads that are still running**. The second point is what makes the system truly peer-to-peer: a client that is still downloading is already a source for anyone who starts after it. In a test, Carol started a 400 MB download when Bob had 394 of its 763 pieces, and she received 225 pieces from him. A request for any other name or path is refused.

6 Tracker synchronisation

Both trackers must give the same answers. The method: do not copy the data, copy the *commands*.

The idea

If two trackers start empty and apply the same commands in the same order, they end up with the same data. This works because the commands are **deterministic**: the result depends only on the current data and the command text. There is no randomness and no clock inside them. (This technique is known as *state machine replication*.)

So whenever a client's command changes something, the tracker that accepted it does three things: applies it, writes it to a log file, and sends it to the other tracker. Commands that change nothing (`list_groups`, a failed login) are not logged and not sent.

The operation log

tracker	number	command (exactly what is replayed)
1	1	CREATE_USER alice pbkdf2-sha256\$600000\$...
1	2	LOGIN alice
1	3	REGISTER_ADDR alice 127.0.0.1:5001 9255...
1	4	CREATE_GROUP movies alice
2	1	CREATE_USER bob pbkdf2-sha256\$600000\$...
2	2	LOGIN bob
2	3	JOIN_GROUP movies bob
1	5	ACCEPT_REQUEST movies bob

Tracker 1 counts its own
operations 1, 2, 3, ...
Tracker 2 counts its own.

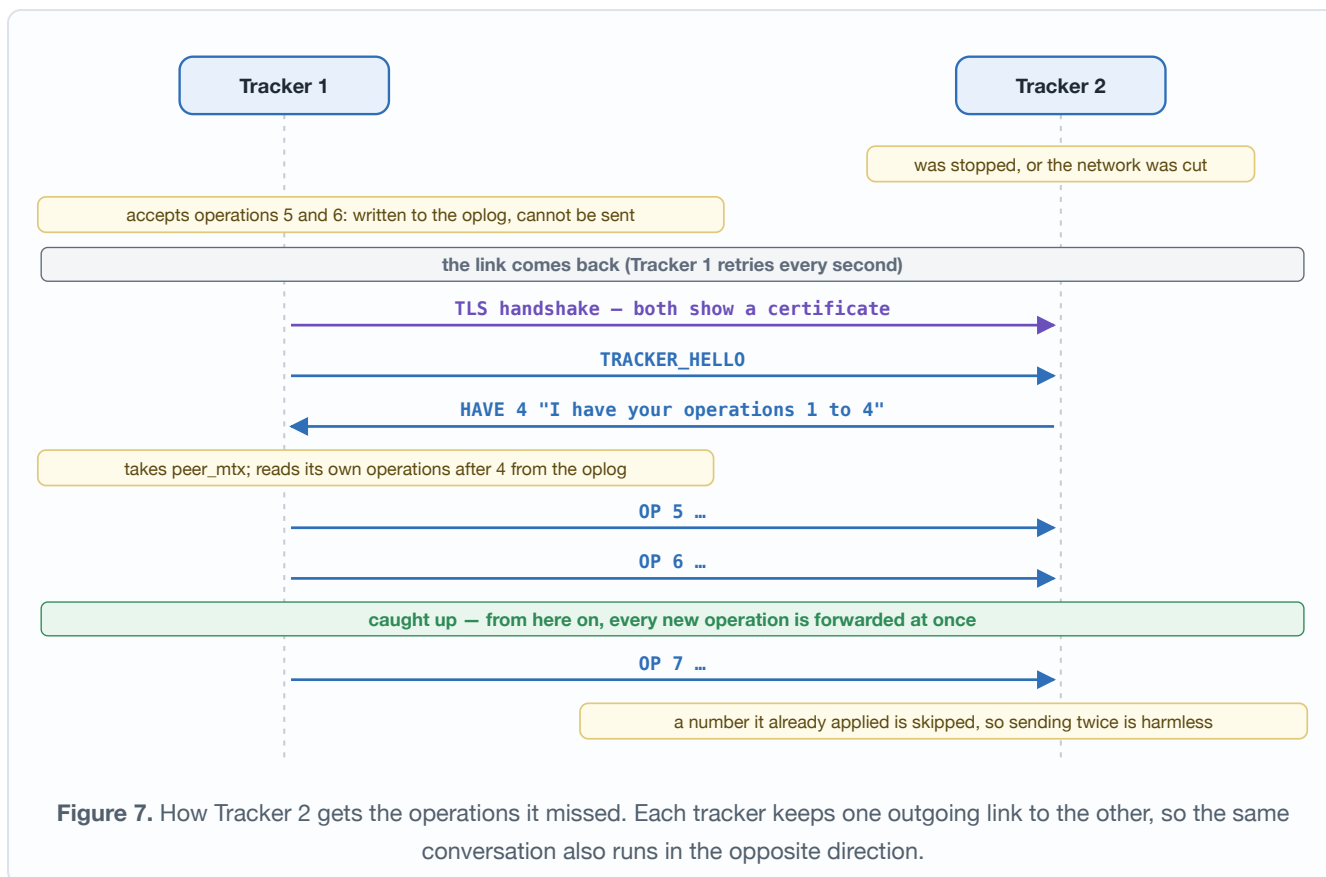
Two jobs of this file:
1. restart → replay it all
2. the other tracker missed
some → read them back
and send them again

Figure 6. `oplog.txt`. Each tracker numbers the operations it accepted from its own clients. The first two columns together identify an operation.

Catching up after the trackers were separated

The interesting case is when one tracker was down, or the network between them was cut, while the other kept working. Sequence numbers solve this. Each tracker needs only three small variables:

Variable	Meaning
<code>my_seq</code>	the number of the last operation I accepted from my own clients
<code>peer_seq</code>	"I have applied all of the other tracker's operations up to this number, with no gaps" — this is what HAVE reports
<code>peer_ahead</code>	operations of the other tracker that I have applied beyond <code>peer_seq</code> (they can arrive out of order after a restart)

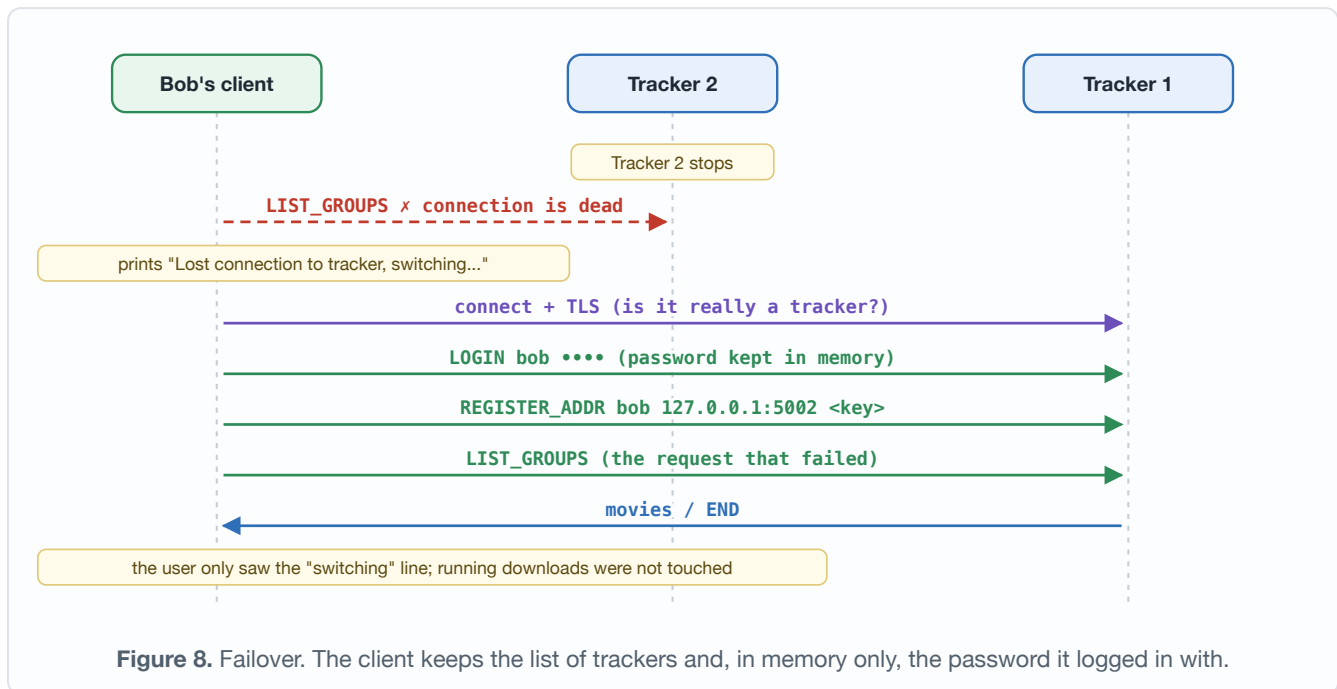


The subtle part: nothing may slip between "resend the old" and "forward the new". While Tracker 1 is resending operations 5 and 6, one of its clients may create operation 7. Without care, 7 could be dropped: too new for the resend list, too early for the live link. The code holds one lock (`peer_mtx`) from the moment it reads the log until the link is marked usable. Every operation is then either in the resend list or forwarded afterwards. In the worst case it is sent twice, and the receiver ignores the copy because it has seen that number.

Restart

A tracker that starts reads `oplog.txt` from the top and applies every line. That rebuilds all its data. Anything newer arrives through the catch-up above. Both trackers append to the same file when they are started from the same folder; every line is written with one `write` call on a file opened in append mode, so lines from the two programs never mix.

When a tracker disappears: the client's side

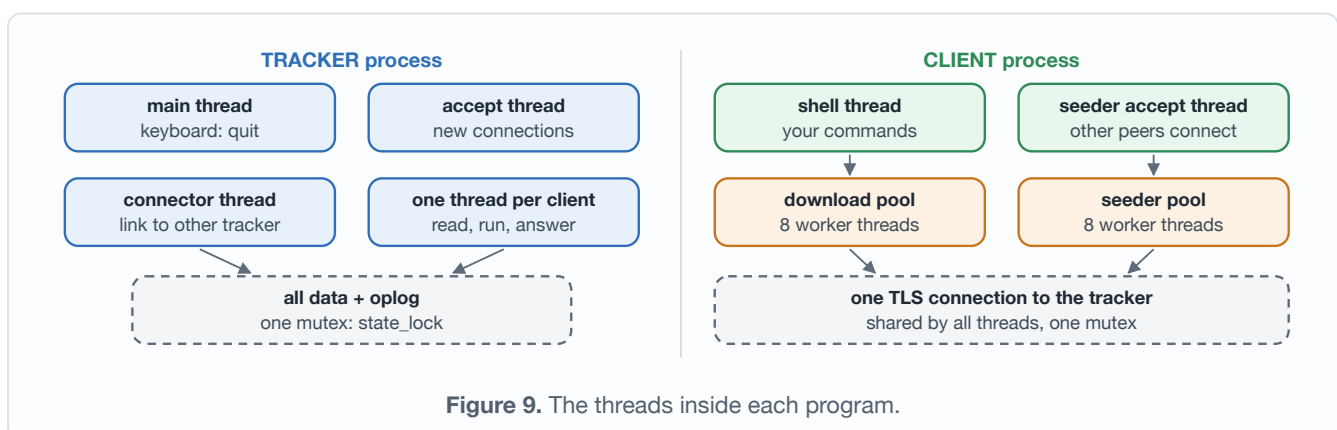


The other tracker already knows the user, because the account was synchronised when it was created. A new login replaces any older session of the same user, which is exactly what this situation needs.

What this design does not solve

If both trackers accept *conflicting* commands at the same instant (for example, two people create the same user name on different trackers), each applies its own first and the other's second, and they can end up different. Solving that needs one tracker to decide the order for both (a leader, as in the Raft algorithm). With two trackers and human-speed commands this is rare, and it is stated openly as a limitation.

7 Threads and locks



Why the two programs use threads differently

- **Tracker: one thread per connected client.** There are few clients (the assignment tests with three), each connection lives long and is mostly idle. One thread per connection is the simplest correct design.

- **Client: fixed thread pools.** A 1 GB download is 2,048 piece requests, and any number of peers may connect to the seeder. Creating a thread for each would be unbounded. A pool of 8 workers puts a hard limit on threads and open sockets, however many downloads are started or peers connect.

The thread pool in a few lines

```
submit(task):  lock;  put task in the queue;  unlock;  wake one worker
worker:        loop: lock;  sleep until there is a task (or we are stopping);
                  take one task;  unlock;  run the task      ← the task runs WITHOUT the lock
destructor:    set "stopping";  wake everybody;  wait for every worker to finish
```

Every lock and what it protects

Mutex	Where	Protects
state_lock	tracker	all tracker data and writes to the log. One big lock: every command is applied as one indivisible step.
peer_mtx	tracker	the connection to the other tracker
threads_mtx	tracker	the count of live threads and the set of open sockets (needed for a clean quit)
g_tracker_mtx	client	the one connection to the tracker, used by the shell, by download threads and by seeder threads
job->mtx	client	the state of one download: queue of pieces, which peers have what, counters
share_mtx , access_mtx	client	the table of shared files; the seeder's short memory of the tracker's answers
console lock	both	everything printed to the terminal: many threads print, and one statement prints as a whole

Why one big lock in the tracker is the right choice here

Commands are tiny (a few map operations) and rare (typed by people). One lock makes every command atomic and makes races impossible to write by accident. Finer locks would add risk for no measurable gain. The one slow thing a tracker does, hashing a password, is deliberately done *before* the lock is taken (section 8).

No deadlocks

A deadlock needs two threads that take the same two locks in opposite orders. Only one place in the code holds two locks at once (peer_mtx , then state_lock , during catch-up) and nowhere are they taken the other way round. Slow work (network transfers, file hashing) is never done while holding a lock, with one deliberate exception: the seeder reads a piece from a running download under that download's lock, so the file cannot be closed underneath it.

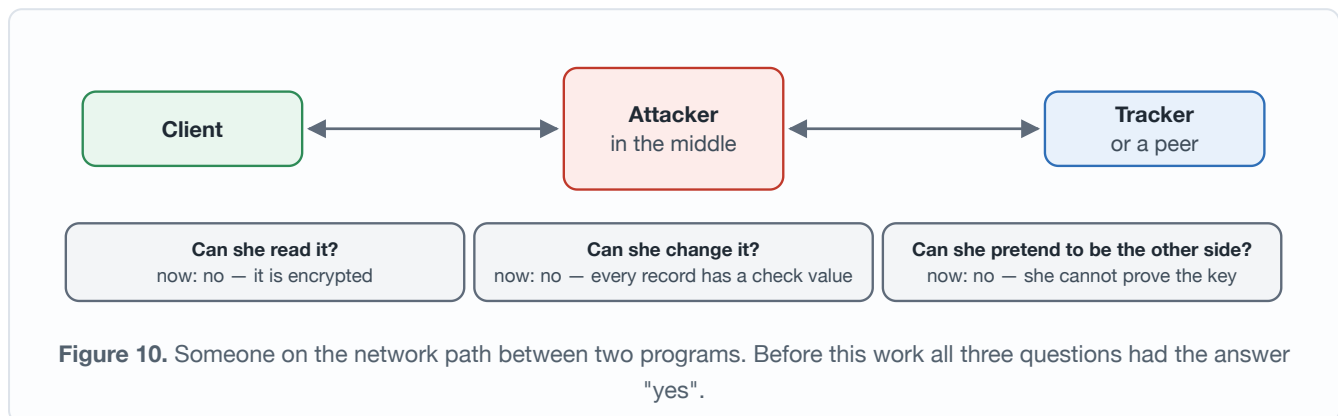
Clean shutdown

quit in the tracker sets a flag, shuts down every open socket (which wakes every thread blocked on one) and waits for the thread count to reach zero. quit in the client cancels downloads, stops the seeder and joins all threads. Threads waiting in accept use poll with a 200 ms limit so they notice the flag. The leak checker reports 0 leaks for both programs.

8 Security (beyond the assignment)

The assignment does not ask for this section's work. It was added afterwards as an extension. Two honest consequences: the programs now need certificate files (made by `make cert`) to start, and the custom protocols of section 3 now travel inside TLS, which comes from the OpenSSL library. Sections 1–7 are unchanged by it apart from three small protocol additions named below.

The threat: a man in the middle



Without protection, anyone on the same network could read passwords, answer in place of a tracker, or change a tracker's reply. The last one is the worst: the piece hashes come from the tracker, so whoever can change that reply can make the SHA-1 checks of section 4 approve a wrong file.

Encryption alone is not enough

If a client encrypts its connection without knowing *who* is at the other end, an attacker can sit in the middle, encrypt separately with each side, and read everything. So each connection must also prove identity. TLS does both, and every connection in the system now uses it. There is no unencrypted mode to fall back to.

Link	How each side knows who the other is
Client → tracker	A tracker must show a certificate signed by the project's own small certificate authority and prove it holds the matching private key. Clients are given the authority's certificate (<code>ca_cert.pem</code>) together with <code>tracker_info.txt</code> .
Tracker → tracker	Both sides show such a certificate. The synchronisation link, whose operations skip all permission checks, is accepted only from a connection that did.
Client → client	Peers have no certificates. Each client makes a fresh key pair when it starts and registers its <i>fingerprint</i> with the tracker. The tracker hands the fingerprint out together with the address. A downloader talks only to a peer that proves it holds that key.

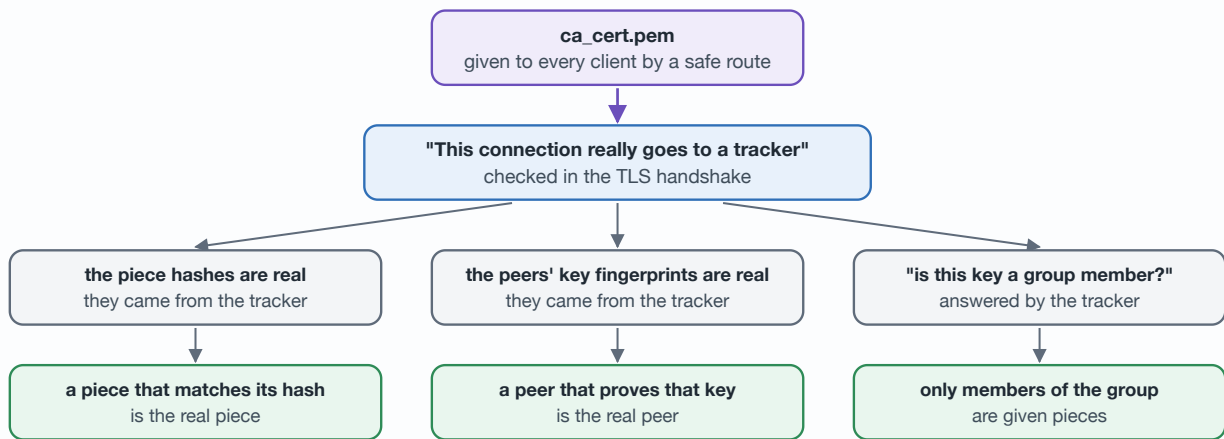


Figure 11. Everything is derived from one file that the client receives by a safe route.

Passwords are never stored

TLS protects a password while it travels. It does nothing about where it rests. Originally the log file contained every password in plain text. Now the tracker turns a password into a **salted, slow hash** the moment it arrives (PBKDF2-HMAC-SHA256, 600,000 rounds, a random salt per user) and keeps only that.

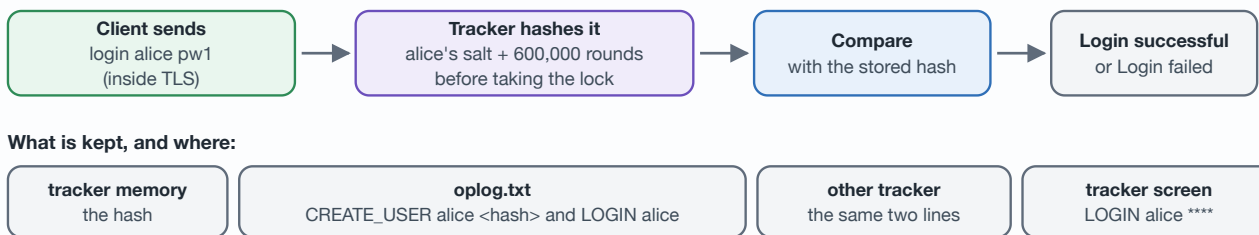
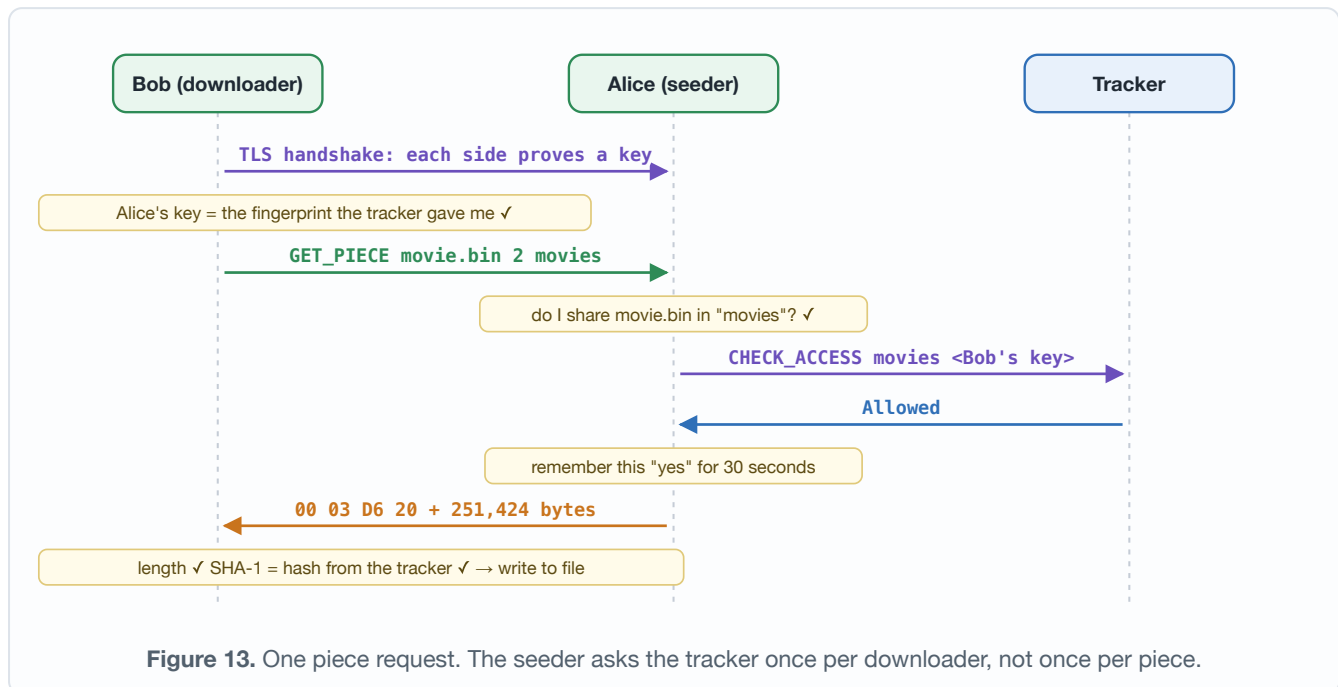


Figure 12. A login. The password itself goes no further than the function that hashes it.

- **Salt** means two users with the same password get different hashes, and a table of pre-computed hashes is useless.
- **Slow** means someone who steals the log must spend 600,000 hash rounds for every guess.
- The hashing takes about 0.15 seconds, so it is done **before** the tracker's lock is taken. Inside the lock it would stall every other client.
- A stolen hash cannot be used to log in: the tracker hashes whatever it is sent.

The seeder checks who is asking

At first a seeder gave pieces to anyone who knew a file name. Now it answers only members of the group the file is shared in.



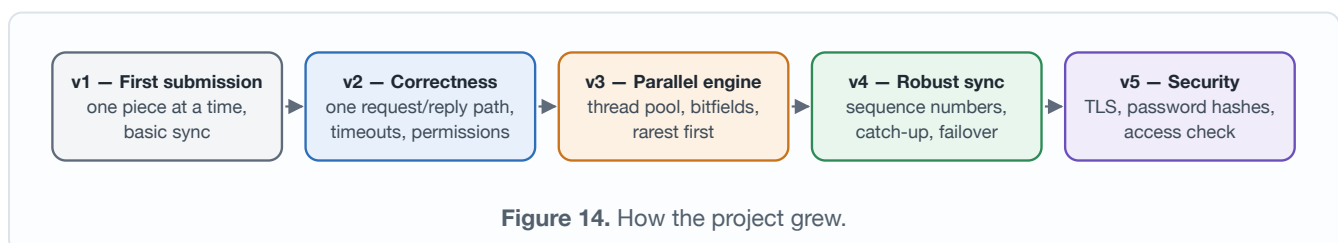
If the tracker cannot be asked, the answer is "no". Every refusal looks the same from outside, so a stranger cannot even find out what a seeder has.

Protocol additions made by this section

REGISTER_ADDR and the peer list carry a key fingerprint; GET_BITFIELD and GET_PIECE end with the group name; a new request CHECK_ACCESS <group> <fingerprint> is answered with Allowed or Denied.

9 Challenges and how they were solved

The first version worked for the simple case and had real problems. Reviewing it critically and fixing it taught more than writing it.



#	Problem	What caused it	Solution
1	Every answer from the tracker was the answer to the <i>previous</i> command	An upload was sent twice and only one line of a multi-line reply was read. The unread lines were then taken as the next reply.	One function (<code>exchange</code>) is the only place that sends a request and reads a reply, and it always reads to <code>END</code> .
2	The seeder would send any file on the disk	It opened whatever path the request named.	A share table: only files this client uploaded or downloaded are served.
3	A silent peer froze a download forever	A timeout was set, but when it fired the code simply tried again.	A timeout is a failure. The piece goes to another peer; repeated failures drop the peer.
4	Downloads used one peer and one piece at a time	The simplest loop was written first.	Thread pool, bitfields, rarest-first, four pieces in flight, least-busy peer.
5	Any client could act as any user	The tracker trusted the name written in the command.	The login is tied to the connection; every command is checked against it.
6	After a restart, the other tracker stopped sending updates	The link was connected once at start-up and never again.	A thread that reconnects every second, with catch-up by sequence number.
7	A user was logged out for no reason after a tracker restart	Two correct features combined badly: a tracker that quits logged its clients out, and catch-up later delivered that logout <i>after</i> the user had logged in on the other tracker.	A tracker that is shutting down does not log its clients out. Found by reading the log of a test, not by a failing test.
8	Adding TLS broke an assumption about threads	One thread sat reading the sync link to notice when it broke, while others wrote on it. Fine for a socket; not allowed for one TLS connection.	The reader watches the raw socket without touching the TLS state (<code>MSG_PEEK</code>).
9	Password hashing would have frozen the tracker	The natural place for it was inside the function that runs under the big lock.	Hash first, take the lock afterwards.
10	Log lines from different threads could mix	Several threads wrote to the terminal with no lock. Every test passed; ThreadSanitizer found it.	One console lock in each program, held for a whole printed statement.

The lesson that came up again and again: a check is only as good as the thing it compares against. The SHA-1 check looked like security, but the hashes arrived over an unprotected connection. Asking "where does this value come from?" found more problems than any amount of testing the normal case.

10 Testing and results

All tests below were run on one machine (Apple M1, macOS) with two trackers and up to three clients, each client driven by typed or piped commands. Downloaded files were compared with the originals byte by byte using `cmp` .

What the assignment asks to be tested

Test	What was observed	Result
Full flow: users, group, join, accept, upload, list, download	Every command gave the expected reply; <code>[C] [movies] movie.bin</code> shown	✓ pass
File sizes: 12 bytes, 3 MB, 200 MB, 1 GB	All copies identical to the original	✓ pass
One file from two peers at once	Pieces arrived from both addresses in the same download	✓ pass
Three downloads at once in one client	All three completed; progress visible while running; prompt stayed usable	✓ pass
A peer's copy is replaced with wrong data	"Hash mismatch" for the pieces from that peer; download completed from the other peer; result identical	✓ pass
Clients on different trackers	A group created on one tracker was listed and joined through the other	✓ pass
A tracker is stopped during a session	The client switched to the other tracker on its next command and kept its login	✓ pass
A tracker starts late, in its own folder	The running tracker printed "resent 12 operations"; the late one then listed everything	✓ pass
Both trackers restarted	State rebuilt from the log; users could log in again	✓ pass
<code>logout</code> stops sharing	The logged-out client was no longer offered as a peer	✓ pass
Memory leaks (macOS <code>leaks --atExit</code>)	0 leaks for the client and for the tracker	✓ pass
An empty file (0 bytes)	Uploaded, listed with size 0, downloaded as an empty file	✓ pass
One of three peers frozen (<code>kill -STOP</code>)	The request to it gave up after the timeout; the download finished from the others	✓ pass
<code>quit</code> while a download is running	The client exited and removed the <code>.part</code> file	✓ pass
A tracker killed with <code>kill -9</code> , then restarted	Clients carried on through the other tracker; the restarted one knew what happened meanwhile	✓ pass
A client that is still downloading used as a source	225 of 763 pieces came from a peer that had not finished	✓ pass
Permission checks: non-member, non-owner, duplicate user, wrong password, second login	Each refused with its own message; the older session was ended	✓ pass
AddressSanitizer + UBSan build, scripted session with 4 clients	18 checks passed, no report	✓ clean
ThreadSanitizer build, same session	18 checks passed, no report (after challenge 10 was fixed)	✓ clean

Measurements: a 1 GB file

What	Measured
upload_file (hash 1 GB, register 2,048 hashes)	2.1 s
download_file, one seeder, including the final whole-file check	7.8 s (about 138 MB/s)
Peak memory of the downloading client	about 12 MB
Peak memory of the seeding client	about 11 MB
Peak memory of a tracker	about 7 MB
Size of the upload message	about 84 KB

Reading these numbers. Both clients ran on the same machine, so this measures the program and the disk, not a real network. TLS was on for every connection.

The important number is the memory: 12 MB for a file of 1,024 MB. It confirms that the file is never loaded as a whole, on either side.

On one machine, downloading from two peers is not faster than from one, because both read the same disk. On a real network the gain depends on the peers' upload speeds.

Security tests (attacking the system)

Attack	What happened	Result
Send a command to the tracker without TLS	Connection refused with a TLS alert; nothing was created	✓ blocked
A relay between client and tracker records everything	The client worked normally through it; the recording contained neither the password nor the word LOGIN	✓ blocked
A fake tracker with a self-made certificate	The client refused the connection and sent it nothing	✓ blocked
Pretend to be the other tracker and inject an operation	Rejected, with and without a self-made certificate	✓ blocked
A fake peer answers on a seeder's address	Dropped before any request was sent to it	✓ blocked
A stranger asks a seeder for a shared file	Refused: with no key, with a key of its own, and with the wrong group	✓ blocked
Search the log and the tracker's screen for passwords	None found; the log holds a hash for account creation and no secret for logins	✓ pass
Log in using a hash copied from the log	Login failed	✓ blocked
Replace a tracker's key while clients keep their files	Clients reconnected with no change on their side	✓ pass

Not tested: a network cut between two *running* trackers (only a stopped or killed tracker was tested), and clients on separate machines. A downloader also does not pick up peers that join after it started; that is a limit of the design, listed in section 11.

11 Limitations and future work

Knowing where a design stops is part of understanding it.

Limitation	Why it is like this	How it could be improved
A new connection (and TLS handshake) for every piece	One request, one reply, close: the simplest protocol that cannot get out of step	Keep one connection per peer and send many requests on it
An interrupted download starts again from zero	The list of pieces already held lives only in memory	Save the bitfield next to the <code>.part</code> file; on restart re-check those pieces and continue
Conflicting commands on both trackers at the same instant can leave them different	Each tracker orders its own operations; nobody orders across both	One tracker decides the order for both (a leader, as in Raft)
The log grows forever	Restart replays the whole history	Write a snapshot of the state from time to time and drop the log before it
A session whose tracker crashed stays "logged in" until that user logs in again	A crashed tracker cannot say goodbye for its clients	A time limit on sessions, renewed by a heartbeat
Bitfields are refreshed only when a download is stuck, and the peer list is fetched once	Enough for this scale. A running download does not use peers that join later, or pieces a peer gained since it was asked.	Peers announce each new piece, as BitTorrent does; ask the tracker for peers again from time to time
A file is identified by its name inside a group	Simple for users	Identify files by their hash
Login attempts are not limited	Not required; each attempt already costs the tracker 0.15 s	Slow down or lock an account after repeated failures
A stolen tracker key works until its certificate expires	There is no list of cancelled certificates	Short-lived certificates or a revocation list
SHA-1 is no longer considered secure against deliberate collisions	The assignment specifies SHA-1	SHA-256: a change in two small functions

If the system had to serve thousands of clients

- **Tracker:** replace one-thread-per-client with an event loop, split the single lock by group, and add snapshots so that restart time does not grow with history.
- **Clients:** reuse connections, limit the number of peers per download, and have peers announce new pieces.

12 References

- B. Cohen, *Incentives Build Robustness in BitTorrent*, 2003 — pieces, piece hashes and rarest-first selection.

- W. R. Stevens, B. Fenner, A. Rudoff, *UNIX Network Programming, Volume 1* — sockets, partial reads and writes, non-blocking connect, `poll`.
- Linux and macOS manual pages: `socket`, `send`, `recv`, `poll`, `pwrite`, `rename`, `setsockopt`, `signal`.
- D. Ongaro, J. Ousterhout, *In Search of an Understandable Consensus Algorithm (Raft)*, 2014 — background for replicating a log of commands and for what leader-based ordering would add.
- OpenSSL documentation: `EVP` digest interface (SHA-1), `SSL_CTX` and `SSL` interfaces, `PKCS5_PBKDF2_HMAC`.
- RFC 8446, *The Transport Layer Security (TLS) Protocol Version 1.3*.
- OWASP, *Password Storage Cheat Sheet* — choice of PBKDF2 and its iteration count.